```
                else
                    relax2(u,*rho[j]);
            }
        }
    }
};
```

CITED REFERENCES AND FURTHER READING:

Brandt, A. 1977, "Multilevel Adaptive Solutions to Boundary-Value Problems," *Mathematics of Computation*, vol. 31, pp. 333–390.[1]

Brandt, A. 1982, in *Multigrid Methods*, W. Hackbusch and U. Trottenberg, eds. (Springer Lecture Notes in Mathematics No. 960) (New York: Springer).[2]

Hackbusch, W. 1985, *Multi-Grid Methods and Applications* (New York: Springer).[3]

Stuben, K., and Trottenberg, U. 1982, in *Multigrid Methods*, W. Hackbusch and U. Trottenberg, eds. (Springer Lecture Notes in Mathematics No. 960) (New York: Springer), pp. 1–176.[4]

Briggs, W.L., Henson, V.E., and McCormick, S. 2000, *A Multigrid Tutorial* (Cambridge, UK: Cambridge University Press).[5]

Trottenberg, U., Oosterlee, C.W., and Schuller, A. 2001, *Multigrid* (Cambridge, MA: Academic Press).[6]

McCormick, S., and Rude, U., eds. 2006, "Multigrid Computing," special issue of *Computing in Science and Engineering*, vol. 8, No. 6 (November/December), pp. 10–62.

Hackbusch, W., and Trottenberg, U. (eds.) 1991, *Multigrid Methods III* (Basel: Birkhäuser).

Wesseling, P. 1992, *An Introduction to Multigrid Methods* (New York: Wiley); corrected reprint 2004 (Philadelphia: R.T. Edwards).

20.7 Spectral Methods

Spectral methods are a very powerful tool for solving PDEs. When they can be used, they are the method of choice if you need high spatial resolution in multi-dimensions. For a second-order accurate finite difference code in three dimensions, increasing the resolution by a factor of 2 in each dimension requires eight times as many grid points, and improves the error typically by a factor of 4. In a spectral code, a similar increase in resolution often gives an improvement of a factor of 10^6. Even for one-dimensional problems, spectral methods will amaze you with their power and efficiency.

Spectral methods work well for smooth solutions. Discontinuities like shocks are bad — don't even try spectral methods. Even mild nonsmoothness (like a discontinuity in some high-order derivative of the solution) can spoil the convergence of spectral methods. (Actually, getting spectral methods to work with discontinuities and shocks is an active research area; see [1] for an introduction.)

The key difference between finite difference methods and spectral methods is that in finite difference methods you approximate the *equation* you are trying to solve, whereas in spectral methods you approximate the *solution* you are trying to find. While finite differencing replaces the continuum equation by an equation on grid points, a spectral method expresses the solution as a truncated expansion in a set

of basis functions:

$$f(x) \simeq f_N(x) = \sum_{n=0}^{N} a_n \phi_n(x) \tag{20.7.1}$$

Different choices of basis functions and methods of computing a_n give different flavors of spectral methods.

20.7.1 Example

We illustrate the idea of spectral methods with an example. Consider the one-sided wave equation (advective equation) in one dimension:

$$\frac{\partial u}{\partial t} = \frac{\partial u}{\partial x} \tag{20.7.2}$$

with periodic boundary conditions on $[0, 2\pi]$ and initial condition

$$u(t = 0, x) \;=\; f(x) \tag{20.7.3}$$

You get the analytic spectral solution by expanding u in a Fourier series,

$$u(t, x) = \sum_{n=-\infty}^{\infty} a_n(t) e^{inx} \tag{20.7.4}$$

Substituting this expansion into equation (20.7.2) gives

$$\frac{da_n}{dt} = ina_n \tag{20.7.5}$$

with solution

$$a_n(t) = a_n(0) e^{int} \tag{20.7.6}$$

You get $a_n(0)$ from the initial condition: Expand

$$f(x) = \sum_{n=-\infty}^{\infty} f_n e^{inx} \tag{20.7.7}$$

from which you see that

$$a_n(0) = f_n \tag{20.7.8}$$

For example, suppose

$$f(x) = \sin(\pi \cos x) \tag{20.7.9}$$

which gives the analytic solution

$$u(t, x) = \sin[\pi \cos(x + t)] \tag{20.7.10}$$

The spectral coefficients in the solution (20.7.4) are

$$\begin{aligned} a_n(0) &= \frac{1}{2\pi} \int_0^{2\pi} \sin(\pi \cos x) e^{-inx} dx \\ &= (-1)^{(n-1)/2} J_n(\pi), \qquad n \text{ odd} \end{aligned} \tag{20.7.11}$$

In a numerical version of this spectral solution, we would truncate the expansion at $n = N$. How well does $u_N(t,x)$ approximate the exact solution? One measure is the root-mean-square error,

$$\begin{aligned} L_2 &= \left[\frac{1}{2\pi}\int_0^{2\pi} |u(t,x) - u_N(t,x)|^2\, dx\right]^{1/2} \\ &= \left[\frac{1}{2\pi}\int_0^{2\pi} \Big|\sum_{|n|>N} a_n(0) e^{inx} e^{int}\Big|^2 dx\right]^{1/2} \\ &= \left[\sum_{|n|>N} |a_n(0)|^2\right]^{1/2} \end{aligned} \tag{20.7.12}$$

Now $J_n(\pi)$ goes to zero exponentially as $n \to \infty$, so the error decreases *exponentially* with N for any $t \geq 0$. This is the key feature of a good spectral method, one you should always strive for. By contrast, a second-order finite difference method has an error that scales as $1/N^2$.

This exponential convergence of spectral methods sets in when one has resolved the main features of the solution. In the above example, the Bessel functions go rapidly to zero once $n \gtrsim \pi$, which corresponds to having about π basis functions per wavelength. On can show that this is a general property of spectral methods [2]. By contrast, second-order accurate finite differencing needs about 20 points per wavelength for 1% accuracy [2]. Moreover, once the solution is resolved, the accuracy improves much more quickly with spectral methods.

There are three properties of the functions e^{inx} that are crucial for this analytic spectral solution, which is just the separation of variables technique:

1. They are a complete set of basis functions.
2. Each basis function by itself obeys the boundary conditions.
3. They are eigenfunctions of the operator in the problem, d/dx.

As we'll see, only property 1 is essential for numerical spectral methods. Spectral methods are not limited to Fourier series — a wide choice of basis functions can be used.

20.7.2 Choice of Basis Functions

You can't simply use Fourier series as basis functions for all problems — it depends on the boundary conditions. Here is a recipe that will take care of 99% of cases you'll encounter:

- If the solution is periodic, use Fourier series.
- If the solution is not periodic and the domain is a square or a cube, or can be mapped to a rectangular region by a simple coordinate transformation, use Chebyshev polynomials along each dimension.
- If the domain is spherical, use spherical harmonics for the angles. In the radial direction, use Chebyshev polynomials for a spherical shell. For a sphere that includes the origin, use the radial basis functions in [8]. These incorporate the correct analytic behavior at the origin and are much better than other choices. They can also be used for cylindrical domains. If the domain is infinite, consult [9,10,4].

Expansions based on for example Chebyshev or Legendre polynomials have the property that their convergence rate is governed by the smoothness of the solution only, not the boundary conditions it satisfies. Fourier expansions, on the other hand, require periodic boundary conditions as well as smoothness for rapid convergence. (These properties are proved, e.g., in [2]. The key point is that basis functions whose convergence rate is independent of the boundary conditions are solutions of singular Sturm-Liouville equations.) It is this independence from the details of the boundary conditions that makes basis functions like Chebyshev polynomials "magical."

Another reason for the popularity of Chebyshev polynomials is that they are really just trigonometric functions whose argument θ has been mapped by $x = \cos\theta$:

$$T_n(x) = \cos(n\theta), \qquad x = \cos\theta \tag{20.7.13}$$

Thus an expansion in Chebyshev polynomials can be evaluated efficiently by the FFT. Moreover, the derivatives of such an expansion can also be evaluated by FFT techniques, as discussed below.

For spherical domains, spherical harmonics are products of Legendre functions in $\cos\theta$ and Fourier series in ϕ. Once again one gets exponential convergence for smooth functions.

20.7.3 Computing the Expansion Coefficients

How do we compute the a_n? There are three basic ways, which can be compared by considering the residual when the expansion (20.7.1) is substituted into the equation you are trying to solve:

1. *Tau method.* Here we require that the a_n be computed so that the boundary conditions are satisfied, and that the residual be orthogonal to as many of the basis functions as possible.
2. *Galerkin method.* In this case you combine the basis functions into a new set, each of which satifies the boundary conditions. Then make the residual orthogonal to as many of the new basis functions as possible. (This is essentially what you do when you separate variables in solving a PDE, as we did for equation 20.7.2. Usually you start with basis functions that already satisfy the boundary conditions individually.)
3. *Collocation* or *pseudospectral method.* As in the tau method, require the boundary conditions to be satisfied, but make the residual *zero* at a set of suitably chosen points.

As we will see, the pseudospectral method has an alternative interpretation that makes it very easy to use. Accordingly, we will only discuss this method, leaving the others to the references.

The big advantage of the pseudospectral method is that it is easy to implement for nonlinear problems. Instead of working with the spectral coefficients, as with the other two methods, you work with the values of the solution at the special grid points associated with the basis functions (typically, the Gaussian quadrature points). These are called the *collocation points.* Often we say we are working with the solution in *physical space* as opposed to in *spectral space*.

A pseudospectral method is an *interpolating* method: Think of the representation

$$y(x) = \sum_{n=0}^{N} a_n \phi_n(x) \tag{20.7.14}$$

as a polynomial that interpolates the solution. Require this interpolating polynomial to be exactly equal to the solution at the $N + 1$ collocation points. If we do things right, then as $N \to \infty$, the errors in between the points tend to zero exponentially fast.

20.7.4 Spectral Methods and Gaussian Quadrature

Recall the formula for Gaussian quadrature (§4.6.1):

$$\int_a^b y(x)w(x)\,dx \approx \sum_{i=0}^{N} w_i\, y(x_i) \tag{20.7.15}$$

Here $w(x)$ is the so-called *weight function* that typically factors out some singular behavior of the integrand, leaving $y(x)$ as a smooth function. The formula is derived by choosing the $2N + 2$ weights and abscissas, w_i and x_i, by requiring that the formula be exact for the polynomials $1, x, x^2, \ldots, x^{2N+1}$. (Don't be confused by the notation: There is no direct relationship between w_i and $w(x)$.) As shown in §4.6, Gaussian quadrature is related to the orthogonal polynomials $\phi_n(x)$ with the given weight function:

$$\langle \phi_n | \phi_m \rangle \equiv \int_a^b \phi_n(x)\phi_m(x)w(x)\,dx = \delta_{mn} \tag{20.7.16}$$

The abscissas x_i turn out to be the $N + 1$ roots of $\phi_{N+1}(x)$, and the weights w_i are given by equation (4.6.9).

We can use Gaussian quadrature to define the discrete inner product of two functions:

$$\langle f | g \rangle_{\mathrm{G}} \equiv \sum_{i=0}^{N} w_i\, f(x_i)g(x_i) \tag{20.7.17}$$

Here the subscript G stands for Gaussian.

An important property of Gaussian quadrature is the *discrete orthogonality relation*

$$\langle \phi_n | \phi_m \rangle_{\mathrm{G}} = \delta_{mn}, \qquad m + n \le 2N + 1 \tag{20.7.18}$$

Proof: Equation (20.7.18) is the Gaussian quadrature version of equation (20.7.16). By assumption, the integrand $\phi_n(x)\phi_m(x)$ of equation (20.7.16) is a polynomial of degree $m + n \le 2N + 1$. But Gaussian quadrature is arranged to integrate polynomials of degree $\le 2N + 1$ exactly. QED.

Now suppose we approximate $y(x)$ by the pseudospectral interpolating polynomial

$$P_N(x) = \sum_{n=0}^{N} \overline{a}_n \phi_n(x) \tag{20.7.19}$$

where the collocation points are chosen to be the Gaussian quadrature points:

$$P_N(x_i) = y(x_i), \qquad i = 0, 1, \ldots, N \tag{20.7.20}$$

This is always possible, since the interpolating polynomial through $N + 1$ points is a polynomial of degree N, and the functions up to $\phi_N(x)$ are a basis for such polynomials. The perhaps unexpected result is that the coefficients $\{\overline{a}_n\}$ of the expansion (20.7.19) are given *exactly* by the Gaussian quadrature

$$\overline{a}_n = \langle y|\phi_n\rangle_{\mathrm{G}} \tag{20.7.21}$$

To see this, take the discrete inner product of both sides of equation (20.7.19) with ϕ_m:

$$\langle P_N|\phi_m\rangle_{\mathrm{G}} = \sum_{n=0}^{N} \overline{a}_n \langle \phi_n|\phi_m\rangle_{\mathrm{G}} \tag{20.7.22}$$

If we use the discrete orthogonality relation (20.7.18), the right-hand side evaluates to $\overline{a}_m$. On the left-hand side, we can replace $P_N(x_i)$ in the Gaussian quadrature by $y(x_i)$ since P_N is the interpolating polynomial. Hence the result follows.

Now comes the key point. The actual spectral expansion of $y(x)$ is

$$y(x) = \sum_{n=0}^{\infty} a_n \phi_n(x) \tag{20.7.23}$$

where the *exact* spectral coefficients are

$$a_n = \langle y|\phi_n\rangle = \int_a^b y(x)\phi_n(x)w(x)\,dx \tag{20.7.24}$$

The pseudospectral expansion coefficients $\overline{a}_n$ are the exact expansion coefficients of $P_N(x)$, the interpolating polynomial (20.7.19). The relation between the exact spectral coefficients and the pseudospectral expansion coefficients follows from equation (20.7.21):

$$\begin{aligned}
\overline{a}_n &= \langle y|\phi_n\rangle_{\mathrm{G}} \\
&= \sum_{m=0}^{\infty} a_m \langle \phi_m|\phi_n\rangle_{\mathrm{G}} \qquad \text{(using equation 20.7.23)} \\
&= \sum_{m=0}^{N} a_m \langle \phi_m|\phi_n\rangle_{\mathrm{G}} + \sum_{m>N} a_m \langle \phi_m|\phi_n\rangle_{\mathrm{G}} \\
&= a_n + \sum_{m>N} a_m \langle \phi_m|\phi_n\rangle_{\mathrm{G}}
\end{aligned} \tag{20.7.25}$$

Thus, since for large N the exact spectral coefficients give an exponentially good approximation to $y(x)$, so do the pseudospectral coefficients. By the way, this is the reason for the name pseudospectral method: We use coefficients that are not the actual spectral coefficients, but are very close to them. From now on we won't bother to distinguish between the two sets of coefficients; we just write a_n for either a_n or $\overline{a}_n$.

The Gaussian quadrature collocation points, the roots of $\phi_{N+1}(x)$, all lie inside the interval (a, b), away from the endpoints. There is another version of Gaussian quadrature that includes the two endpoints of the interval. This is called Gauss-Lobatto quadrature, and the collocation points are the Gauss-Lobatto points (§4.6.4). These points are as effective as the ordinary Gaussian points, and are more convenient when you need to impose boundary conditions at the endpoints.

As a slight digression, you may be under the mistaken impression that the only advantage of Gaussian integration over integration with equally spaced points is that its degree of exactness is $2N + 1$ as opposed to N, the maximum you can get with only the $N + 1$ weights at your disposal. In fact, however, the main advantage of Gaussian integration is that it converges exponentially with N for smooth functions. You can see this explicitly from the above formulas by setting $m = 0$ in equation (20.7.21):

$$\bar{a}_0 = \phi_0 \sum_{i=0}^{N} w_i y(x_i) \tag{20.7.26}$$

where ϕ_0 is a constant. But this converges exponentially to the expression given by equation (20.7.24):

$$a_0 = \phi_0 \int_a^b y(x) w(x)\, dx \tag{20.7.27}$$

as claimed.

How do Fourier series fit into this discussion? After all, the collocation points are equally spaced (usually $x_j = 2\pi j/N$, $j = 0, \ldots, N-1$). But in fact these are the correct collocation points if we think of Fourier series as interpolating $y(x)$ by a *trigonometric* polynomial. The corresponding Gaussian quadrature (using the equally spaced points) is the midpoint rule, and the Gauss-Lobatto quadrature, which includes the endpoints, is the trapezoidal rule. The textbooks tell you that the midpoint and trapezoidal rules are low-order methods. This is true for arbitrary functions. But if you apply them to *periodic* functions (§5.8.1), or functions that go rapidly to zero at infinity (§4.5 and §13.11), they are in fact exponentially convergent, like any self-respecting Gaussian quadrature method should be.

20.7.5 Cardinal Functions

You can write *any* polynomial interpolation formula for a function $f(x)$ as

$$P_N(x) = \sum_{i=0}^{N} f(x_i) C_i(x) \tag{20.7.28}$$

where the $C_i(x)$ are called *cardinal functions*. They are polynomials of degree N that satisfy

$$C_i(x_j) = \delta_{ij} \tag{20.7.29}$$

i.e., $C_i(x)$ is 1 at the ith collocation point and 0 at all the others.

One explicit representation of cardinal functions comes from the formula for Lagrange interpolation (see equation 3.2.1):

$$C_i(x) = \prod_{\substack{j=0 \\ j \neq i}}^{N} \frac{x - x_j}{x_i - x_j} \tag{20.7.30}$$

If you substitute this in equation (20.7.28), it is just the Lagrange interpolation formula. Each choice of basis functions implies a corresponding choice of collocation points x_j, and so a corresponding choice of cardinal functions by equation (20.7.30).

There are other equivalent ways of writing $C_i(x)$. For example, if $\phi_n(x)$ is a set of orthogonal polynomials, and the collocation points are the zeros of $\phi_{N+1}(x)$ (Gaussian quadrature points), then $C_i(x)$ is almost $\phi_{N+1}(x)$, except $\phi_{N+1}(x)$ vanishes at *all* the grid points. Since near $x = x_i$

$$\phi_{N+1}(x) = \phi_{N+1}(x_i) + (x - x_i)\phi'_{N+1}(x_i) + \cdots \tag{20.7.31}$$

we get the cardinal function by dividing out the zero at $x = x_i$:

$$C_i(x) = \frac{\phi_{N+1}(x)}{(x - x_i)\phi'_{N+1}(x_i)} \tag{20.7.32}$$

In practice you don't need to know any of the formulas like equations (20.7.30) or (20.7.32). The books in the references have formulas for the $C_i(x)$ for all the standard basis functions if you are curious. What you do need are the derivatives of the cardinal functions, the *differentiation matrices* (see below).

You might be nervous about using very high-order polynomial interpolation to represent your solution, especially if you've ever encountered the *Runge phenomenon*: If the grid points are *equally spaced*, then the error in $P_N(x)$ can tend to infinity as $N \to \infty$. What happens is that the error shows up near the endpoints of the interval — the middle is fine. The fix is to make the points more concentrated toward the endpoints, which is exactly what choosing the Gaussian points does. This is the same reason why Chebyshev approximation often works when polynomial approximation fails, as was discussed in §5.8.1.

20.7.6 Spectral vs. Grid Point Representation

Let's contrast the representations of the solution of

$$\mathcal{L}y = f \tag{20.7.33}$$

in spectral space and in physical space. Assume that $\mathcal{L}$ is a linear differential operator for simplicity.

Spectral Space	**Physical Space**
$y(x) = \sum_{n=0}^{N} a_n \phi_n(x)$	$y(x) = \sum_{j=0}^{N} y_j C_j(x)$
$\sum_{n=0}^{N} a_n \mathcal{L}\phi_n(x) = f(x)$	$\sum_{j=0}^{N} y_j \mathcal{L}C_j(x) = f(x)$
Impose at collocation points only:	
$\sum_{n=0}^{N} a_n \mathcal{L}\phi_n(x_j) = f(x_j)$	$\sum_{j=0}^{N} y_j \mathcal{L}C_j(x_i) = f(x_i)$
i.e., $La = f$, where $L_{jn} = \mathcal{L}\phi_n(x_j)$	i.e., $L^{(c)}y = f$, where $L^{(c)}_{ij} = \mathcal{L}C_j(x_i)$

The two representations are related as follows: To go from grid point values to spectral coefficients you project $y(x)$ along each basis function:

$$\begin{aligned} a_i &= \langle \phi_i \mid y \rangle \\ &= \sum_j w_j \phi_i(x_j) y_j \qquad \text{(doing the integral by Gaussian quadrature)} \end{aligned} \tag{20.7.34}$$

That is,

$$\mathbf{a} = \mathbf{M} \cdot \mathbf{y}, \qquad \text{where} \quad M_{ij} = \phi_i(x_j) w_j \tag{20.7.35}$$

Thus the relation in spectral space $\mathbf{L} \cdot \mathbf{a} = \mathbf{f}$ becomes $\mathbf{L} \cdot \mathbf{M} \cdot \mathbf{y} = \mathbf{f}$. But in physical space $\mathbf{L}^{(c)} \cdot \mathbf{y} = \mathbf{f}$, so

$$\mathbf{L}^{(c)} = \mathbf{L} \cdot \mathbf{M} \tag{20.7.36}$$

with inverse

$$\mathbf{L} = \mathbf{L}^{(c)} \cdot \mathbf{M}^{-1} \tag{20.7.37}$$

Note also that equation (20.7.35) implies

$$\mathbf{y} = \mathbf{M}^{-1} \cdot \mathbf{a} \tag{20.7.38}$$

Since $y = \sum a_n \phi_n$, we see that $\mathbf{M}^{-1}$ is the matrix that sums the spectral series to get the grid point values, i.e.,

$$M^{-1}_{ij} = \phi_j(x_i) \tag{20.7.39}$$

You can check that these relations are all consistent:

$$\begin{aligned}
(\mathbf{M} \cdot \mathbf{M}^{-1})_{ij} &= \sum_k M_{ik} M^{-1}_{kj} \\
&= \sum_k [\phi_i(x_k) w_k][\phi_j(x_k)] \\
&= \langle \phi_i | \phi_j \rangle_{\mathrm{G}} \\
&= \delta_{ij} \qquad \text{(by discrete orthogonality)}
\end{aligned} \tag{20.7.40}$$

In practice, the transformations (20.7.35) and (20.7.38) are often done with FFTs for Fourier or Chebyshev basis functions if N is large. For simple programs, just do matrix multiplication.

20.7.7 Differentiation Matrices

We've seen above that the key ingredient in the pseudospectral method is to form

$$L^{(c)}_{ij} = \mathcal{L} C_j(x_i) \tag{20.7.41}$$

which involves taking derivatives of the cardinal functions at the collocation points. Consider the first derivative ∂_x. You then need the matrix

$$D^{(1)}_{ij} = \partial_x C_j(x_i) \tag{20.7.42}$$

This quantity can be computed ahead of time and stored. Then, to compute the vector of first derivatives at the grid points, just do a matrix multiplication:

$$\frac{\partial y}{\partial x} \quad \longleftrightarrow \quad \sum_{j=0}^{N} D^{(1)}_{ij} y_j \tag{20.7.43}$$

Similarly one can define the second derivative matrix $D^{(2)}_{ij}$, and so on.

The matrix multiplication in equation (20.7.43) requires $O(N^2)$ operations. For Fourier basis functions e^{ikx}, one can compute the derivative alternatively as follows:

$$\begin{aligned} y &\xrightarrow{\text{FFT}} a \\ a &\longrightarrow ika \\ ika &\xrightarrow{\text{inverse FFT}} y' \end{aligned}$$

For Chebyshev basis functions, there is a simple $O(N)$ recurrence relation in the middle step to get the coefficients for the derivative from the coefficients for the function (see equation 5.9.2). Thus the procedure is $O(N \log N)$. However, it is typically faster than the $O(N^2)$ matrix multiplication only for $N \gtrsim 16 - 128$, depending on the computer. So just use matrix multiplication for simple programs.

It is worth pointing out that this idea of using recurrence relations to evaluate operators in spectral space is much more general than the simple example of derivatives of Chebyshev functions. It is important for efficient production codes when the operators consist of derivatives times simple powers of the coordinates. See the references for details.

20.7.8 Computing Differentiation Matrices

There are several options for computing differentiation matrices:

1. Derive the formulas by differentiating the Lagrange polynomial representation (20.7.30).
2. Differentiate the basis function representation (20.7.32).
3. Look up the explicit formulas that have been derived for the various basis functions in books, e.g. Chapter 2 of [3].
4. Use the routine given below, based on the routine in [6]. This algorithm computes any order of differentiation matrix given only a set of collocation points $\{x_i\}$.

Obviously, the last choice is the easiest. However, it does have the potential drawback for high-precision work that roundoff error can be larger than necessary. If this is a problem, see [7].

weights.h

`void weights(const Doub z, VecDoub_I &x, MatDoub_O &c)`

Compute the differentiation matrices for pseudospectral collocation. Input are `z`, the location where the matrices are to be evaluated, and `x[0..n]`, the set of `n+1` grid points. On output, `c[0..n][0..m]` contains the weights at grid locations `x[0..n]` for derivatives of order `0..m`. The element `c[j][k]` contains the weight to be applied to the function value at `x[j]` when the kth derivative is approximated by the set of $n+1$ collocation points `x`. Note that the elements of the zeroth derivative matrix are returned in `c[0..n][0]`. These are just the values of the cardinal functions, i.e., the weights for interpolation.

```
{
    Int n=c.nrows()-1;
    Int m=c.ncols()-1;
    Doub c1=1.0;
    Doub c4=x[0]-z;
    for (Int k=0;k<=m;k++)
        for (Int j=0;j<=n;j++)
            c[j][k]=0.0;
    c[0][0]=1.0;
    for (Int i=1;i<=n;i++) {
        Int mn=MIN(i,m);
```

```
        Doub c2=1.0;
        Doub c5=c4;
        c4=x[i]-z;
        for (Int j=0;j<i;j++) {
            Doub c3=x[i]-x[j];
            c2=c2*c3;
            if (j == i-1) {
                for (Int k=mn;k>0;k--)
                    c[i][k]=c1*(k*c[i-1][k-1]-c5*c[i-1][k])/c2;
                c[i][0]=-c1*c5*c[i-1][0]/c2;
            }
            for (Int k=mn;k>0;k--)
                c[j][k]=(c4*c[j][k]-k*c[j][k-1])/c3;
            c[j][0]=c4*c[j][0]/c3;
        }
        c1=c2;
    }
}
```

Typical usage of the `weights` routine to compute first- and second-order derivative matrices is

```
VecDoub x(n);
MatDoub c(n,3),d1(n,n),d2(n,n);
for (j=0;j<n;j++)
    x[j]= ...
for (i=0;i<n;i++) {
    weights(x[i],x,c);
    for (j=0;j<n;j++) {
        d1[i][j]=c[j][1];
        d2[i][j]=c[j][2];
    }
}
```

20.7.9 A Note on Interpolation

Often you want to evaluate the solution at points that are not the collocation points. This requires an interpolation. To preserve the full spectral accuracy, you want to use all the information in the solution. However, it is not necessary to transform the solution to spectral space and then evaluate the representation (20.7.1) at the desired point, e.g., by Clenshaw's method. Just use the interpolation formula (20.7.28). A simple way to do this is to use the above routine, which will return the interpolation weights $C_i(x_k)$ for any set of target points x_k when `m`, the second dimension of `c` in the code, is zero. So interpolating to a set of points can again be done as a matrix multiplication.

20.7.10 Pseudospectral Collocation as a Finite Difference Method

Consider finite difference approximations for d/dx at the center of an equally spaced grid, for example

$$\begin{aligned} hf'(x) &= -\tfrac{1}{2}f(x-h) + \tfrac{1}{2}f(x+h) + O(h^2) \\ &= \tfrac{1}{12}f(x-2h) - \tfrac{2}{3}f(x-h) + \tfrac{2}{3}f(x+h) - \tfrac{1}{12}f(x+2h) + O(h^4) \\ &= \quad \ldots \end{aligned} \tag{20.7.44}$$

For centered differences like these, the limit as $N \to \infty$ of the weights (coefficients of f) is finite. But for one-sided approximations, or partially one-sided approximations, the weights diverge [5]. Since one has to use such approximations near the endpoints of the grid, it's not surprising that high-order finite difference approximations have large errors near the boundaries.

But suppose the grid points are not equally spaced. In particular, suppose they are closer together near the endpoints, like the Gaussian quadrature points. Then the finite difference approximation is convergent as $N \to \infty$.

The pseudospectral method gives the exact derivative of the interpolating polynomial that passes through the data at the $N + 1$ grid points. You would get the same result for a finite difference method that uses *all* $N + 1$ grid points, This follows from the uniqueness of the interpolating polynomial, a polynomial of degree N through all $N + 1$ points.

With this point of view, think of a pseudospectral method as a way to find high-order numerical approximations to derivatives at grid points. Then, just like finite difference methods, satisfy the equation you want to solve at the grid points.

20.7.11 Variable Coefficients and Nonlinearities

Suppose you have a term like $\sinh(x)\, y(x)$ in your equation. No need to expand $\sinh(x)$ in basis functions — just multiply $\sinh(x)$ by y at each collocation point. Similarly, nonlinear terms like y^2 are evaluated directly using the values at the collocation points. This is the big advantage over the tau and Galerkin methods — handling nonlinearities in physical space rather than spectral space is much easier.

20.7.12 A Worked Example

Here is a simple one-dimensional example, taken from Appendix B of [5]. Consider the equation

$$y'' + y' - 2y + 2 = 0, \qquad -1 \le x \le 1, \tag{20.7.45}$$

$$y(-1) = y(1) = 0 \tag{20.7.46}$$

The exact solution is

$$y(x) = 1 - \frac{e^x \sinh 2 + e^{-2x} \sinh 1}{\sinh 3} \tag{20.7.47}$$

Let's make an expansion in Chebyshev polynomials with $N = 4$:

$$y = \sum_{n=0}^{4} a_n T_n(x) \tag{20.7.48}$$

Choose the collocation points to be

$$x_i = -\cos \frac{i\pi}{4}, \qquad i = 0, \ldots, 4 \tag{20.7.49}$$

These are the Gauss-Lobatto points associated with Chebyshev polynomials, i.e., they include the endpoints. We always include the endpoints when we want to impose Dirichlet boundary conditions, that is, function values on the boundaries. Using

one of the methods for finding differentiation matrices, we get

$$[D^{(1)}y]_i = \begin{bmatrix} -\frac{11}{2} & 4+2\sqrt{2} & -2 & 4-2\sqrt{2} & -\frac{1}{2} \\ -1-\frac{1}{2}\sqrt{2} & \frac{1}{2}\sqrt{2} & \sqrt{2} & -\frac{1}{2}\sqrt{2} & 1-\frac{1}{2}\sqrt{2} \\ \frac{1}{2} & -\sqrt{2} & 0 & \sqrt{2} & -\frac{1}{2} \\ -1+\frac{1}{2}\sqrt{2} & \frac{1}{2}\sqrt{2} & -\sqrt{2} & -\frac{1}{2}\sqrt{2} & 1+\frac{1}{2}\sqrt{2} \\ \frac{1}{2} & -4+2\sqrt{2} & 2 & -4-2\sqrt{2} & \frac{11}{2} \end{bmatrix} \begin{bmatrix} y_0 \\ y_1 \\ y_2 \\ y_3 \\ y_4 \end{bmatrix} \tag{20.7.50}$$

and

$$[D^{(2)}y]_i = \begin{bmatrix} 17 & -20-6\sqrt{2} & 18 & -20+6\sqrt{2} & 5 \\ 5+3\sqrt{2} & -14 & 6 & -2 & 5-3\sqrt{2} \\ -1 & 4 & -6 & 4 & -1 \\ 5-3\sqrt{2} & -2 & 6 & -14 & 5+3\sqrt{2} \\ 5 & -20+6\sqrt{2} & 18 & -20-6\sqrt{2} & 17 \end{bmatrix} \begin{bmatrix} y_0 \\ y_1 \\ y_2 \\ y_3 \\ y_4 \end{bmatrix} \tag{20.7.51}$$

Requiring that the differential equation hold at the interior collocation points x_k, $k = 1, 2, 3$, uses the middle three rows of these matrices. Enforcing the boundary conditions $y_0 = y_4 = 0$ means we don't need the first and last columns. So equation (20.7.45) gives

$$\begin{bmatrix} -16+\frac{1}{2}\sqrt{2} & 6+\sqrt{2} & -2-\frac{1}{2}\sqrt{2} \\ 4-\sqrt{2} & -8 & 4+\sqrt{2} \\ -2+\frac{1}{2}\sqrt{2} & 6-\sqrt{2} & -16-\frac{1}{2}\sqrt{2} \end{bmatrix} \begin{bmatrix} y_1 \\ y_2 \\ y_3 \end{bmatrix} = \begin{bmatrix} -2 \\ -2 \\ -2 \end{bmatrix} \tag{20.7.52}$$

with solution

$$\begin{bmatrix} y_1 \\ y_2 \\ y_3 \end{bmatrix} = \begin{bmatrix} \frac{101}{350}+\frac{13}{350}\sqrt{2} \\ \frac{13}{25} \\ \frac{101}{350}-\frac{13}{350}\sqrt{2} \end{bmatrix} \tag{20.7.53}$$

The exact solution (20.7.47) gives for example $y(x = 0) = 0.52065$, compared with $y_2 = 0.52000$. Not bad for five grid points! The real point, however, is that the error is about 10^{-16} for $N = 16$. With a second-order finite difference scheme, the error would go down by only a factor of 10 or so with this increase in N.

20.7.13 Multidimensional Spectral Methods

For a time-dependent problem, the simplest approach is the *method of lines*. Expand the solution as

$$y(t, x) = \sum_j C_j(x) y_j(t) \tag{20.7.54}$$

where now the coefficients y_j are functions of time. Then

$$\left.\frac{\partial y}{\partial t}\right|_i = \dot{y}_i, \qquad \left.\frac{\partial y}{\partial x}\right|_i = \sum_j D_{ij}^{(1)} y_j, \quad \text{etc.} \tag{20.7.55}$$

You get a system of ODEs in t for the y_j, which you can solve in the standard way. Runge-Kutta is a good method to start with.

Problems with two or three spatial dimensions are usually handled by making expansions along each dimension separately:

$$u(x,y,z) = \sum_{ijk} u_{ijk} C_i(x) C_j(y) C_k(z) \tag{20.7.56}$$

Elliptic equations give simultaneous algebraic equations for the coefficients that are typically solved with iterative methods because of the large number of variables. See [11] for an example and references to the literature.

CITED REFERENCES AND FURTHER READING:

Hesthaven, J., Gottlieb, S., and Gottlieb, D. 2007, *Spectral Methods for Time-Dependent Problems* (New York: Cambridge University Press), Chapter 9.[1]

Gottlieb, D., and Orszag, S.A. 1977, *Numerical Analysis of Spectral Methods: Theory and Applications* (Philadelphia: S.I.A.M.).[2] [A classic, and still somewhat useful.]

Canuto, C., Hussaini, M.Y., Quarteroni, A., and Zang, T.A. 1988, *Spectral Methods in Fluid Dynamics* (Berlin: Springer).[3] [Standard reference for fluid dynamics applications, but applicable to other areas.]

Boyd, J.P. 2001, *Chebyshev and Fourier Spectral Methods*, 2nd ed. (New York: Dover Publications). Available at `http://www-personal.engin.umich.edu/~jpboyd`.[4] [Best single book: complete, and not too formal.]

Fornberg, B. 1996, *A Practical Guide to Pseudospectral Methods* (New York: Cambridge University Press).[5] [Good for getting started, but not for large-scale problems.]

Fornberg, B. 1998, "Calculation of Weights in Finite Difference Formulas," *SIAM Review* vol. 40, pp. 685–691.[6]

Baltensperger, R., and Trummer, M.R. 2003, "Spectral Differencing with a Twist," *SIAM Journal on Scientific Computing*, vol. 24, pp. 1465–1487.[7]

Matsushima, T., and Marcus, P.S. 1995, "A Spectral Method for Polar Coordinates," *Journal of Computational Physics* vol. 120, pp. 365–374.[8]

Matsushima, T., and Marcus, P.S. 1997, "A Spectral Method for Unbounded Domains," *Journal of Computational Physics* vol. 137, pp. 321–345.[9]

Rawitscher, G.H. 1991, "Accuracy Analysis of a Bessel Spectral Function Method for the Solution of Scattering Equations," *Journal of Computational Physics* vol. 94, pp. 81–101.[10]

Pfeiffer, H.P., Kidder, L.E., Scheel, M.A., and Teukolsky, S.A. 2003, "A Multidomain Spectral Method for Solving Elliptic Equations," *Computer Physics Communications*, vol. 152, pp. 253–273.[11]

Bjørhus, M. 1995, "The ODE Formulation of Hyperbolic PDEs Discretized by the Spectral Collocation Method," *SIAM Journal on Scientific Computing*, vol. 16, pp. 542–557. [Describes a good algorithm for hyperbolic equations.]